

Assignment #A: 递归回溯、 🌲 (3/4)

Updated 2203 GMT+8 Nov 3, 2025

2025 fall, Compiled by 窦兆文 元培学院

说明：

1. 解题与记录：

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge，Codeforces，LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **提交安排：**提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交：**如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

T51.N皇后

backtracking, <https://leetcode.cn/problems/n-queens/>

思路：

代码：

代码运行截图（至少包含有"Accepted"）

M22275: 二叉搜索树的遍历

<http://cs101.openjudge.cn/practice/22275/>

思路：

给定前序遍历 [root, 左子树元素..., 右子树元素...]:

第一个元素是根节点

找到第一个大于根的元素,它是右子树的起点

根之后、右子树之前的都是左子树

递归处理左右子树

后序遍历 = 左子树后序 + 右子树后序 + 根

代码：

```
def preorder_to_postorder(n, preorder):
    if not preorder:
        return []
    root = preorder[0]
    right_start = -1
    for i in range(1, len(preorder)):
        if preorder[i] > root:
            right_start = i
            break
    if right_start == -1:
        left_subtree = preorder[1:]
        right_subtree = []
    else:
        left_subtree = preorder[1:right_start]
        right_subtree = preorder[right_start:]
    left_post = preorder_to_postorder(len(left_subtree), left_subtree)
    right_post = preorder_to_postorder(len(right_subtree), right_subtree)
    return left_post + right_post + [root]

n = int(input())
preorder = list(map(int, input().split()))
postorder = preorder_to_postorder(n, preorder)
print(' '.join(map(str, postorder)))
```

代码运行截图（至少包含有"Accepted"）

#50877989提交状态

状态: Accepted

— ... —

M25145: 猜二叉树（按层次遍历）

<http://cs101.openjudge.cn/practice/25145/>

思路:

根据中序和后序遍历重建二叉树

后序遍历最后一个元素是根节点

在中序遍历中找到根节点位置

根节点左边是左子树,右边是右子树

递归构建左右子树

层次遍历

使用队列进行BFS

从根节点开始,逐层访问

代码:

```

class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def build_tree(inorder, postorder):
    if not inorder or not postorder:
        return None

    root_val = postorder[-1]
    root = TreeNode(root_val)

    root_idx = inorder.index(root_val)

    left_inorder = inorder[:root_idx]
    right_inorder = inorder[root_idx + 1:]

    left_postorder = postorder[:len(left_inorder)]
    right_postorder = postorder[len(left_inorder):-1]

    root.left = build_tree(left_inorder, left_postorder)
    root.right = build_tree(right_inorder, right_postorder)

    return root

def level_order(root):
    if not root:
        return ""

    result = []
    queue = [root]

    while queue:
        node = queue.pop(0)
        result.append(node.val)

        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)

    return ''.join(result)

```

```
n = int(input())
for _ in range(n):
    inorder = input().strip()
    postorder = input().strip()

    root = build_tree(inorder, postorder)
    print(level_order(root))
```

代码运行截图（至少包含有"Accepted"）

#50878021提交状态

状态: Accepted

源代码

T20576: printExp（逆波兰表达式建树）

<http://cs101.openjudge.cn/practice/20576/>

思路:

代码

代码运行截图（至少包含有"Accepted"）

T04080:Huffman编码树

greedy, <http://cs101.openjudge.cn/practice/04080/>

思路:

使用贪心算法:

每次选择权值最小的两个节点合并

合并后的新节点权值为两个节点权值之和

将新节点放回集合中
重复直到只剩一个节点

代码

```
import heapq

n = int(input())
weights = list(map(int, input().split()))

heap = weights[:]
heapq.heapify(heap)

total = 0

while len(heap) > 1:
    w1 = heapq.heappop(heap)
    w2 = heapq.heappop(heap)

    merged = w1 + w2
    total += merged

    heapq.heappush(heap, merged)

print(total)
```

代码运行截图（至少包含有"Accepted"）



#50878046提交状态

状态: Accepted

源代码

M04078: 实现堆结构

<http://cs101.openjudge.cn/practice/04078/>

要求手搓堆实现。

思路：

手动实现最小堆：

数据结构：

使用数组存储完全二叉树

父节点索引 i , 左子节点 $2i+1$, 右子节点 $2i+2$

父节点索引 $= (i-1)//2$

核心操作：

插入 (push):

将元素添加到数组末尾

上浮调整(sift_up): 与父节点比较, 若更小则交换, 直到满足堆性质

删除最小值 (pop):

返回根节点(最小值)

将最后一个元素移到根位置

下沉调整(sift_down): 与子节点比较, 与最小子节点交换, 直到满足堆性质

堆性质:

每个节点的值 $\leq$ 其子节点的值

代码：

```

class MinHeap:
    def __init__(self):
        self.heap = []

    def push(self, val):
        self.heap.append(val)
        self._sift_up(len(self.heap) - 1)

    def pop(self):
        if len(self.heap) == 1:
            return self.heap.pop()

        min_val = self.heap[0]
        self.heap[0] = self.heap.pop()
        self._sift_down(0)
        return min_val

    def _sift_up(self, idx):
        while idx > 0:
            parent = (idx - 1) // 2
            if self.heap[idx] < self.heap[parent]:
                self.heap[idx], self.heap[parent] = self.heap[parent], self.heap[idx]
                idx = parent
            else:
                break

    def _sift_down(self, idx):
        n = len(self.heap)
        while True:
            smallest = idx
            left = 2 * idx + 1
            right = 2 * idx + 2

            if left < n and self.heap[left] < self.heap[smallest]:
                smallest = left
            if right < n and self.heap[right] < self.heap[smallest]:
                smallest = right

            if smallest != idx:
                self.heap[idx], self.heap[smallest] = self.heap[smallest], self.heap[idx]
                idx = smallest
            else:
                break

```



```
n = int(input())
heap = MinHeap()

for _ in range(n):
    operation = list(map(int, input().split()))
    op_type = operation[0]

    if op_type == 1:
        u = operation[1]
        heap.push(u)
    elif op_type == 2:
        print(heap.pop())
```

代码运行截图（至少包含有"Accepted"）

#50878058提交状态

状态: **Accepted**

2. 学习总结和个人收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2025fall每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。

通过本周的学习，我掌握了经典的树和堆数据结构算法：包括N皇后问题的回溯法、二叉搜索树和普通二叉树的遍历转换、表达式树的构建与括号处理、哈夫曼树求最优编码、以及手动实现堆的上浮下沉操作。这些算法体现了递归、贪心、分治等核心思想，是解决复杂问题的基础工具。